(`spring.cloud.gateway.httpclient.response-timeout`).

When a circuit breaker switches to the open state, we'll want at least to degrade the service level gracefully and make the user experience as pleasant as possible. I'll show you how to do that in the next section.

### *9.2.3 Defining fallback REST APIs with Spring WebFlux*

When we added the `CircuitBreaker` filter to `catalog-route`, we defined a value for the `fallbackUri` property to forward the requests to the `/catalog-fallback` endpoint when the circuit is in an open state. Since the `Retry` filter is also applied to that route, the fallback endpoint will be invoked even when all retry attempts fail for a given request. It's time to define that endpoint.

As I mentioned in previous chapters, Spring supports defining REST endpoints either using `@RestController` classes or router functions. Let's use the functional way of declaring the fallback endpoints.

In a new `com.polarbookshop.edgeservice.web` package in the Edge Service project, create a new `WebEndpoints` class. Functional endpoints in Spring WebFlux are defined as routes in a `RouterFunction<ServerResponse>` bean, using the fluent API provided by `RouterFunctions`. For each route, you need to define the endpoint URL, a method, and a handler.

**Listing 9.9 Fallback endpoints for when the Catalog Service is down**